Name of Student: Minisha Singh	**PRN: 25070122292**
Semester: III	**Year : AY 2026-27**
Subject: Programming with Java	
ASSIGNMENT No : 1	
TITLE : Java Environment Setup and Basic Program Execution	

Problem Statement

Develop an application based on Java environment setup, JDK/JRE/JVM, variables, methods and constructors. and demonstrate the implementation of the concepts through suitable examples.

Learning Objectives

Understand the theoretical concepts, implement them in Java programs, analyse outputs, and apply the concepts to solve practical programming problems.

Programme

```
public class EmployeeDemo {


   String name;

   int id;

   double salary;


   EmployeeDemo(String n, int i, double s) {

      name = n;

      id = i;

      salary = s;

   }
```

```
    void displayInfo() {
        System.out.println("Name   : " + name);
        System.out.println("ID     : " + id);
        System.out.println("Salary : Rs." + salary);
    }

    double calculateBonus(double percent) {
        return salary * percent / 100;
    }

    public static void main(String args[]) {

        int totalEmployees = 2;
        System.out.println("Total Employees: " + totalEmployees);
        System.out.println("----------------------------");

        EmployeeDemo e1 = new EmployeeDemo("Rohan", 101, 45000.0);
        e1.displayInfo();
        System.out.println("Bonus (10%) : Rs." + e1.calculateBonus(10));

        System.out.println("----------------------------");

        EmployeeDemo e2 = new EmployeeDemo("Priya", 102, 52000.0);
        e2.displayInfo();
        System.out.println("Bonus (10%) : Rs." + e2.calculateBonus(10));
    }
}
```

Output

```
Total Employees: 2
---------------------------
Name   : Rohan
ID     : 101
Salary : Rs.45000.0
Bonus (10%) : Rs.4500.0
---------------------------
Name   : Priya
ID     : 102
Salary : Rs.52000.0
Bonus (10%) : Rs.5200.0

=== Code Execution Successful ===
```

Conclusion

This program demonstrates the core building blocks of Java — environment setup (JDK, JRE, JVM), variables, methods, and constructors — through a simple employee management example. The JDK/JRE/JVM setup enables writing, compiling, and running Java code, while variables store employee data, the constructor initializes it at object creation, and methods (`displayInfo()`, `calculateBonus()`) process and display that data. Together, these concepts form the foundation for building structured, object-oriented Java applications.

Questions

1. Explain the main concept used in Java environment setup, variables and methods.

Ans : **Java Environment Setup:** Process of installing JDK (development tools), JRE (runtime), and JVM (executes bytecode), plus setting PATH so Java commands work in terminal — enables writing, compiling, and running Java programs.

Variables: Named memory locations used to store data of a specific type (e.g., `int`, `double`, `String`); declared as `dataType name = value;`.

Methods: Reusable blocks of code that perform a specific task, defined with a return type, name, and optional parameters, and executed when called.

2. What are the advantages of using Java variables and methods?

Ans : **Advantages of using Variables:**

1. Store and reuse data instead of hardcoding values repeatedly.
2. Make code readable and self-explanatory (meaningful names like `age`, `marks`).
3. Allow values to change dynamically during program execution.
4. Support different data types, so memory is used efficiently (e.g., `int` vs `double`).
5. Enable calculations and logic to work on stored data rather than fixed constants.

Advantages of using Methods:

1. **Reusability** – write code once, call it as many times as needed.
2. **Modularity** – breaks a large program into smaller, manageable tasks.
3. **Easier debugging** – errors are isolated to a specific method, not the whole program.
4. **Improved readability** – `getGrade()` is clearer than repeating if-else logic everywhere.
5. **Abstraction** – you can call a method without knowing its internal implementation.
6. **Reduces code duplication** – less code, fewer chances of mistakes.
7. **Supports maintenance** – changes made in one place (inside the method) reflect everywhere it's used.

3. How can this implementation be improved?

Ans : Based on the `EmployeeDemo` program, here's how it can be improved:

1. **Encapsulation** – make `name`, `id`, `salary` `private`, and add getter/setter methods instead of accessing them directly.
2. **Input validation** – check for invalid values (negative salary, empty name) inside the constructor.
3. **Use `this` keyword** – write `this.name = name;` style in the constructor for clarity when parameter names match variable names.
4. **`toString()` method** – return formatted employee details automatically instead of a separate `displayInfo()` call.
5. **User input support** – use the `Scanner` class to let the user enter employee details at runtime instead of hardcoding them.
6. **ArrayList of employees** – use `ArrayList<EmployeeDemo>` to manage multiple employees instead of separate objects (`e1`, `e2`).
7. **Static counter** – add a `static` variable to auto-count total employees created, instead of manually setting `totalEmployees = 2`.
8. **Exception handling** – wrap risky operations (like input parsing) in `try-catch` blocks.
9. **Separate class file** – keep `EmployeeDemo` class and `main()` in separate files for better structure.

10. **Add method overloading** – e.g., a `calculateBonus()` version with no parameter that uses a default bonus percentage.

4. What are the real-world applications?

Ans : **Real-world applications:**

1. **Variables & Methods** – banking (balance, transactions), e-commerce (price, discount calculation), games (score, health).
2. **Constructors** – auto-setting up user profiles, orders, or game characters the moment they're created.
3. **This student-record structure** → used in school/college systems, HR systems, hospital patient records, library management.
4. **JDK/JRE/JVM setup** – required for any Java app (Android, banking software, enterprise systems); JVM enables "write once, run anywhere."